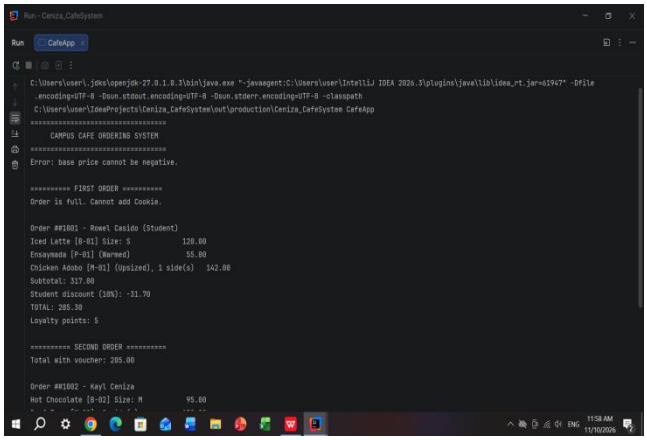
CAMPUS CAFE ORDERING SYSTEM

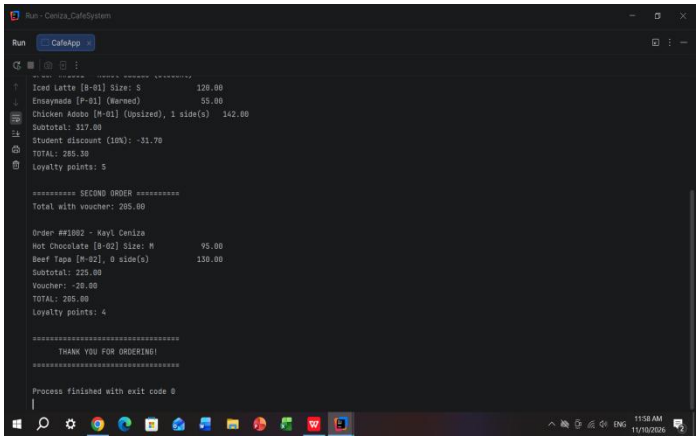
NAME: Kayl Ceniza
Rowel Casido
DATE: 11/10/26
COURSE/SECTION: BSINT/D
SUBJECT: OOP

Concept Explanation:

1. **If Statement** – I used if statements to check conditions, such as calculating the warm fee for pastries and determining the additional size fee for beverages.
2. **If-Else Statement** – I used if-else statements in the MenuItem, Beverage, and Order classes to calculate prices, check item sizes, and validate orders.
3. **Classes and Objects** – I created seven classes: MenuItem, Beverage, Pastry, Meal, Customer, Order, and CafeApp. These classes represent the different parts of the cafe ordering system.
4. **Attributes and Behaviors** – Each class contains attributes and methods. For example, the Customer class stores customer information, while the Order class contains methods for adding items and computing the total.
5. **Object Interaction** – Objects interact with one another in the program. The Order object holds Customer and MenuItem objects, allowing the program to manage orders and print receipts.
6. **Java Naming Conventions** – I followed Java naming conventions by using PascalCase for class names, camelCase for methods and variables, and UPPER_SNAKE_CASE for constants.
7. **Default Constructor** – I used default constructors in classes such as MenuItem, Beverage, Pastry, Meal, and Customer to initialize objects without requiring specific values.
8. **Parameterized Constructor** – I used parameterized constructors to initialize objects with specific information, such as customer names and menu item details.
9. **Constructor Overloading** – I applied constructor overloading in the Customer class by creating three constructors with different parameters to allow different ways of creating customer objects.
10. **Public, Private, and Protected Access Modifiers** – I used public methods to access class functionalities, private fields to protect data, and protected members to allow access within related classes.
11. **Getters and Setters** – I used getters and setters to retrieve and update object information while checking that the entered values were valid.
12. **Data Hiding** – I applied data hiding by declaring fields as private and controlling access through public methods. This helps protect important information from being directly changed.
13. **Method Overloading** – I used method overloading in the Customer class through different addPoints() methods and in the Order class through different computeTotal() methods.
14. **Method Overriding** – I applied method overriding in the Beverage, Pastry, and Meal classes by providing their own implementations of the computePrice() and getDescription() methods inherited from the parent class.
15. **Abstraction** – I used an abstract class named MenuItem with an abstract computePrice() method. This allows the subclasses to provide their own price calculation.

PROGRAM OUTPUT





Challenges Encountered and Solutions

While completing this laboratory exercise, I encountered some challenges in understanding and applying the different Object-Oriented Programming concepts. One of the challenges was implementing constructor overloading, method overloading, and method overriding correctly. I also had difficulty managing the interaction between different classes and making sure that the price calculations were accurate.

To solve these problems, I reviewed the purpose of each OOP concept, checked my constructors and methods, and corrected errors in my code. I also tested the program using different customer orders to make sure that the calculations and validation worked properly.

Overall, this activity helped me improve my understanding of Java programming and how Object-Oriented Programming concepts can be used to create an organized and functional program.